ADR 001: Captura e distribuição de alterações com CDC

PostgreSQL + Debezium + Kafka Connect + Apache Kafka

Data: 16/10/2026

Autores: Rony Alissom Leviske Oenning | Mariana de Moraes Orsi

1. Contexto

A ShopVibe precisa acompanhar alterações nos dados operacionais, especialmente nos pedidos, e disponibilizá-las para processamento com baixa latência. A arquitetura já possui PostgreSQL, Kafka, Kafka Connect e um conector Debezium para PostgreSQL registrado. O objetivo é evitar depender exclusivamente de consultas periódicas ao banco para descobrir novos registros ou alterações.

2. Decisão

Utilizar Change Data Capture (CDC) com Debezium para capturar alterações do PostgreSQL e encaminhá-las ao Apache Kafka por meio do Kafka Connect. O PostgreSQL registra as alterações no Write-Ahead Log (WAL); com a configuração adequada de replicação lógica, o Debezium pode convertê-las em eventos publicados em tópicos Kafka. O Kafka será o ponto de distribuição para os consumidores downstream, incluindo o conector de saída para o MinIO.

3. Alternativas consideradas

- **Consultas periódicas (polling):** Consultar as tabelas em intervalos definidos. É relativamente simples, mas pode introduzir atrasos, gerar consultas repetidas e exigir lógica para identificar registros novos ou modificados.
- **Publicação de eventos pela aplicação:** A aplicação envia um evento a cada operação. Pode ser apropriado, mas exige implementar e manter essa integração no código da aplicação e tratar a consistência entre a gravação no banco e a publicação.
- **CDC com Debezium:** Captura alterações a partir do log do banco e integra-se ao Kafka Connect. É adequado quando se deseja desacoplar a aplicação da distribuição de eventos.

4. Justificativa

CDC com **Debezium** foi escolhido porque se encaixa nas stacks já configuradas e permite distribuir alterações do banco como eventos sem exigir que cada componente consulte diretamente o **PostgreSQL**. O **Kafka** separa a captura dos dados de seus consumidores, permitindo que o armazenamento e futuros processos analíticos evoluam de forma independente.

5. Consequências e validação

- **Benefícios:** captura contínua de alterações; integração padronizada com Kafka; possibilidade de adicionar consumidores sem criar integrações diretas com o banco.
- **Custos e limitações:** exige configurar replicação lógica, permissões e conectividade; requer monitoramento do conector e do atraso de captura; consumidores devem considerar eventos repetidos, ordenação por chave e formatos de evento.

- **Condição de validação:** o conector estar registrado não comprova, por si só, que o fluxo está funcionando. O grupo deverá inserir ou alterar um registro de teste no PostgreSQL e confirmar a chegada do evento ao tópico Kafka.